# Manual de Instalação e Configuração do Backend — Gestão de Clínica Veterinária

| Campo | Valor |
|---|---|
| Grupo | G08 |
| Projeto | Gestão de Clínica Veterinária |
| Artefacto | Manual de Instalação e Configuração do backend |
| Ficheiro | `m3-manual-instalacao-configuracao-v01.pdf` (fonte: `m3-manual-instalacao-configuracao-v01.md`) |
| Versão | v01 |
| Milestone | m3 |
| Issue | I038 — Elaborar o Manual de Instalação e Configuração do backend (GitHub #67) |
| Sprint | S05 |
| Executor | André Filipe Moreira Mesquita |
| Revisor | Gonçalo Magalhães Ribeiro |
| Auditor | Ivan Cristiano Silva Mendes |
| Versão do backend | `develop` no commit `df5b5f2` (2026-11-04) |
| Pasta | `/projeto/04-artefactos-tecnicos/04.17-manual-de-instalacao-e-configuracao` |

## Histórico de versões

| Versão | Data | Autor | Descrição |
|---|---|---|---|
| v01 | 2026-11-04 | André Filipe Moreira Mesquita | Versão inicial: pré-requisitos, obtenção do código, base de dados (SQL Server 2025 local ou em Docker), migrações, dados de demonstração, configuração por ambiente, arranque, verificação, reposição e problemas frequentes. Inclui as correções dos desvios D1 (leitura de palavras-passe no Windows PowerShell 5.1) e D2 (mudança de pasta antes dos passos 3.2 e 4.1) encontrados na validação num ambiente limpo (critério 4 da I038). |

## 1. Objetivo e âmbito

Este manual permite a qualquer elemento do grupo, ou ao docente, instalar e pôr a funcionar o backend da m3 num computador sem nada do projeto: obter o código, criar a base de dados SQL Server 2025 com as migrações da I062, carregar os dados fictícios de demonstração da I061, configurar os segredos sem os guardar no repositório, arrancar a API e verificar que funciona. Explica também como repor o ambiente e como a configuração muda entre os ambientes do projeto.

O manual foi escrito para **Windows 10/11**, a plataforma de desenvolvimento do grupo. Os comandos são de PowerShell. O ambiente público de avaliação no Azure tem procedimento próprio (secção 9.1).

## 2. Pré-requisitos

### 2.1. Obrigatórios

| Software | Versão | Como verificar | Observações |
|---|---|---|---|
| Windows | 10 ou 11 (64 bits) | `winver` | |

| Software | Versão | Como verificar | Observações |
|---|---|---|---|
| .NET SDK | **10.0.100 ou superior na série 10.0** | `dotnet --version → 10.0.x` | Fixado em `src/backend/global.json` (`10.0.100`, `rollForward: latestFeature`). Usado pelo grupo com o 10.0.403. Inclui o runtime ASP.NET Core 10 |
| Ferramenta de migrações `dotnet-ef` | **10.0.13** (igual à versão do EF Core do projeto) | `dotnet ef --version` | Instalação no passo 1.3 |
| SQL Server | **SQL Server 2025** (17.x), edição Developer ou Express, **ou** a imagem Docker `mcr.microsoft.com/mssql/server:2025-CU9-ubuntu-22.04` | `SELECT @@VERSION;` | A imagem Docker é a mesma da pipeline e dos testes de integração |
| `sqlcmd` | SQL Server Command Line Utilities (`sqlcmd` ODBC 18) ou `go-sqlcmd` | `sqlcmd -?` | Usado pelos scripts de dados de demonstração (I061) |
| Git | 2.40 ou superior | `git --version` | |
| PowerShell | Windows PowerShell 5.1 ou PowerShell 7 | `$PSVersionTable.PSVersion` | Os comandos deste manual funcionam nos dois. Não use `Read-Host -MaskInput`: no 5.1 não dá erro, mas mostra a palavra-passe no ecrã |

### 2.2. Opcionais

| Software | Versão | Para quê |
|---|---|---|
| Docker Desktop | Atual, com contentores Linux | SQL Server em Docker (secção 4, opção B) e testes de integração (Testcontainers 4.16.0) |
| Node.js e Newman | Node.js 20 ou superior; Newman 6.2.1 | Executar a Collection Postman (secção 7.3) |
| Visual Studio 2026 ou JetBrains Rider | Compatível com .NET 10 | Desenvolvimento; não é necessário para instalar |
| SQL Server Management Studio ou Azure Data Studio | Atual | Consultar a base de dados |

### 2.3. Informação de que vai precisar

- **Palavras-passe das contas de demonstração:** não estão no repositório. Estão no ficheiro `m3-credenciais-avaliacao-v01.md` da submissão (`/projeto/00-indice-e-submissao`), entregue à parte. Só são necessárias para iniciar sessão (secção 7.2).
- Se usar SQL Server com autenticação SQL (por exemplo, em Docker): uma palavra-passe forte para o login `sa`, escolhida por si.

## 3. Obter o código e preparar as ferramentas

**Passo 1.1 — Clonar o repositório** numa pasta à sua escolha:

```
git clone https://github.com/LEI-LDS-2026-2027-G08/LEI-Grupo-de-LDS-UTPORTO.git
cd LEI-Grupo-de-LDS-UTPORTO
git checkout develop
```

Resultado esperado: a pasta `src/backend` contém `ClinicaVeterinaria.sln`.

**Passo 1.2 — Confirmar o SDK**, na pasta `src/backend`:

```
cd src\backend
dotnet --version
```

Resultado esperado: `10.0.x`. Se aparecer outra versão ou um erro sobre o `global.json`, instale o .NET SDK 10 (problema P1).

**Passo 1.3 — Instalar o `dotnet-ef`:**

```
dotnet tool install --global dotnet-ef --version 10.0.13
dotnet ef --version
```

Resultado esperado: `Entity Framework Core .NET Command-line Tools 10.0.13`. Se já estiver instalado noutra versão, use `dotnet tool update --global dotnet-ef --version 10.0.13`.

**Passo 1.4 — Compilar a solução:**

```
dotnet restore ClinicaVeterinaria.sln
dotnet build ClinicaVeterinaria.sln --configuration Release --no-restore
```

Resultado esperado: `Build succeeded` com `0 Warning(s)` e `0 Error(s)`. A compilação trata os avisos como erros (`TreatWarningsAsErrors` no `Directory.Build.props`, quality gate QG-01).

## 4. Base de dados SQL Server 2025

Escolha **uma** das opções.

### Opção A — SQL Server 2025 instalado no computador (autenticação do Windows)

**Passo 2.A.1 —** Instale o SQL Server 2025 (Developer ou Express) com a instância por omissão (`MSSQLSERVER`) e a autenticação do Windows. Instale também o `sqlcmd` (SQL Server Command Line Utilities ou `go-sqlcmd`).

**Passo 2.A.2 —** Confirme a ligação:

```
sqlcmd -S localhost -E -C -Q "SELECT @@VERSION;"
```

Resultado esperado: uma linha que começa por `Microsoft SQL Server 2025`.

A cadeia de ligação desta opção, usada nos passos 3.1 e 4.2, é:

```
Server=localhost;Database=ClinicaVeterinaria;Trusted_Connection=True;TrustServerCertificate=True
```

Com uma instância com nome (por exemplo, `SQLEXPRESS`), use `Server=localhost\SQLEXPRESS` em vez de `Server=localhost`.

### Opção B — SQL Server 2025 em Docker (autenticação SQL)

**Passo 2.B.1 —** Com o Docker Desktop a correr, crie o contentor (a palavra-passe tem de ter pelo menos 8 caracteres, com maiúsculas, minúsculas, algarismos e símbolos):

```
$sec = Read-Host -AsSecureString "Palavra-passe para o login sa"
$env:SQL_SA = [Runtime.InteropServices.Marshal]::PtrToStringBSTR([Runtime.InteropServices.Marsha
l]::SecureStringToBSTR($sec))
Remove-Variable sec
docker run --name sql-clinica -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=$env:SQL_SA" -p 1433:1433 -d
mcr.microsoft.com/mssql/server:2025-CU9-ubuntu-22.04
```

A palavra-passe não aparece no ecrã ao ser escrita (`-AsSecureString` funciona no Windows PowerShell 5.1 e no PowerShell 7). A variável `$env:SQL_SA` só existe nesta janela do PowerShell: use a mesma janela até ao passo 4.2.

**Passo 2.B.2 —** Aguarde cerca de 20 segundos e confirme:

```
docker logs sql-clinica | Select-String "SQL Server is now ready for client connections"
```

Resultado esperado: a linha é encontrada.

**Passo 2.B.3 —** A cadeia de ligação desta opção é a seguinte; é usada nos passos 3.1 e 4.2 e **não deve ser guardada em ficheiros do repositório**:

```
Server=localhost,1433;Database=ClinicaVeterinaria;User Id=sa;Password=<palavra-passe do sa>;TrustSe
rverCertificate=True;Encrypt=True
```

## 5. Criar a base de dados, aplicar as migrações e carregar os dados

**Passo 3.1 — Criar a base e aplicar as migrações da I062.** Na pasta `src\backend` (onde ficou no passo 1.2), o script de reposição compila a solução, apaga a base indicada se existir, cria-a com as migrações e confirma que não há alterações ao modelo por migrar:

*Opção A:*

```
powershell -ExecutionPolicy Bypass -File .\scripts\reposicao-base-dados.ps1 -ConnectionString "Serv
er=localhost;Database=ClinicaVeterinaria;Trusted_Connection=True;TrustServerCertificate=True"
```

*Opção B (com a cadeia do passo 2.B.3):*

```
powershell -ExecutionPolicy Bypass -File .\scripts\reposicao-base-dados.ps1 -ConnectionString "Serv
er=localhost,1433;Database=ClinicaVeterinaria;User Id=sa;Password=$env:SQL_SA;TrustServerCertificat
e=True;Encrypt=True"
```

Resultado esperado: `[1/4]` a `[4/4]` sem erros e `Reposicao concluida com sucesso.` (numa instalação nova, o passo `[2/4]` indica que a base ainda não existe, o que é normal). A base `ClinicaVeterinaria` tem as 16 tabelas do modelo, mais a `__EFMigrationsHistory`, e os dados de referência das migrações: a clínica «Clínica Veterinária G08», as espécies, as raças e a duração da consulta.

**Passo 3.2 — Carregar os dados fictícios de demonstração da I061.** Primeiro, mude da pasta `src\backend` para a pasta dos dados de demonstração:

```
cd ..\..\04-artefactos-tecnicos\04.13-scripts-de-base-de-dados\m3-v02\dados-demonstracao
```

Resultado esperado: `dir` mostra o `executar-dados.ps1`. Depois, execute:

*Opção A:*

```
powershell -ExecutionPolicy Bypass -File .\executar-dados.ps1 -Servidor "localhost" -BaseDados "Cli
nicaVeterinaria" -ConfiarCertificado
```

*Opção B:*

```
$env:SQLCMDPASSWORD = $env:SQL_SA
powershell -ExecutionPolicy Bypass -File .\executar-dados.ps1 -Servidor "localhost,1433" -BaseDados
"ClinicaVeterinaria" -Utilizador "sa" -ConfiarCertificado
Remove-Item Env:SQLCMDPASSWORD
```

Resultado esperado: o script corre o `02-dados-demonstracao.sql` duas vezes (a segunda com `0 linha(s) inserida(s)` em todas as tabelas, provando que pode ser reexecutado) e termina com **«Verificação dos dados de demonstração: 28 verificações, 28 OK, 0 falhadas.»** Fica um registo `dados-execucao-AAAAMMDD-HHMMSS.log` na pasta atual: **não o adicione ao repositório**.

Dados carregados: 10 contas (uma por perfil, uma bloqueada, Auxiliar com e sem apoio, três Clientes), 4 animais, 5 marcações em estados diferentes, um registo clínico e o boletim de vacinas. Todos os dados são fictícios (nomes com «(demonstração)», e-mails do domínio `.test`).

## 6. Configuração

### 6.1. Definições da API

A API lê a configuração do `appsettings.json`, do `appsettings.{Ambiente}.json`, dos *user secrets* (em Development) e das variáveis de ambiente, por esta ordem de prioridade crescente. Nas variáveis de ambiente, `__` (dois underscores) substitui o `:` (por exemplo, `Jwt__Key` corresponde a `Jwt:Key`).

| Definição | Obrigatória | Valor por omissão | Descrição |
|---|---|---|---|
| `ConnectionStrings:ClinicaDb` | **Sim** | No `appsettings.json`: SQL Server local, base `ClinicaVeterinaria`, autenticação do Windows. **No ambiente Development, o `appsettings.Development.json` deixa-a vazia**, por isso tem de estar nos *user secrets* (passo 4.2) | Cadeia de ligação ao SQL Server |
| `Jwt:Key` | **Sim** (segredo) | — (vazia) | Chave de assinatura dos tokens, em Base64, com pelo menos 32 bytes. Sem ela, a API não arranca |
| `Jwt:Issuer`, `Jwt:Audience` | Sim | `ClinicaVeterinaria.Api`, `ClinicaVeterinaria.Clients` | Emissor e audiência dos tokens |
| `Jwt:ExpirationHours` | Sim | `8` | Validade dos tokens |
| `LimpezaTokensRevogados:IntervaloMinutos` | Sim | `60` | Intervalo da limpeza dos tokens revogados expirados |
| `Cors:AllowedOrigins` | Não | Vazio (qualquer origem) | Origens autorizadas do frontend web |
| `Swagger:Ativo` | Não | `false` | Ativa o Swagger fora do ambiente Development (I104) |
| `BaseDados:AplicarMigracoesNoArranque` | Não | `false` | Aplica as migrações ao arrancar a API; usado só no ambiente público (I104) |

O ficheiro `src/backend/ClinicaVeterinaria.Api/appsettings.Example.json` mostra a estrutura sem segredos.

### 6.2. Segredos — nunca no repositório

**Passo 4.1 — Gerar e guardar a chave JWT** nos *user secrets* (ficam no seu perfil do Windows, fora da pasta do projeto). Primeiro, volte da pasta dos dados de demonstração para a pasta `src\backend`; depois, gere e guarde a chave:

```
cd ..\..\..\..\src\backend
$bytes = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
$chave = [Convert]::ToBase64String($bytes)
dotnet user-secrets set "Jwt:Key" $chave --project .\ClinicaVeterinaria.Api\ClinicaVeterinaria.Api.
```

```
csproj
Remove-Variable chave, bytes
```

Resultado esperado: `Successfully saved Jwt:Key to the secret store.`

**Passo 4.2 — Guardar a cadeia de ligação nos *user secrets*.** Obrigatório nas duas opções: o perfil `http` arranca em Development, e o `appsettings.Development.json` deixa a cadeia vazia.

*Opção A:*

```
dotnet user-secrets set "ConnectionStrings:ClinicaDb" "Server=localhost;Database=ClinicaVeterinari
a;Trusted_Connection=True;TrustServerCertificate=True" --project .\ClinicaVeterinaria.Api\ClinicaVe
terinaria.Api.csproj
```

*Opção B:*

```
dotnet user-secrets set "ConnectionStrings:ClinicaDb" "Server=localhost,1433;Database=ClinicaVeteri
naria;User Id=sa;Password=$env:SQL_SA;TrustServerCertificate=True;Encrypt=True" --project .\Clinica
Veterinaria.Api\ClinicaVeterinaria.Api.csproj
```

Resultado esperado: `Successfully saved ConnectionStrings:ClinicaDb to the secret store.`

Para confirmar: `dotnet user-secrets list --project .\ClinicaVeterinaria.Api\ClinicaVeterinaria.Api.csproj` (mostra os valores: não partilhe o resultado).

Regras do grupo: as chaves JWT, as palavras-passe e as cadeias de ligação reais nunca são escritas em ficheiros versionados nem em mensagens; os ficheiros `http-client.env.json.user` e os registos `dados-execucao-*.log` não se adicionam ao repositório.

## 7. Arrancar e verificar

### 7.1. Arrancar a API

**Passo 5.1 —** Na pasta `src/backend`:

```
dotnet run --project .\ClinicaVeterinaria.Api\ClinicaVeterinaria.Api.csproj --launch-profile http
```

Resultado esperado: a consola mostra `Now listening on: http://localhost:5159` e `Application started`. O perfil `http` usa o ambiente Development, com o Swagger ativo. Deixe esta janela aberta; para parar, use Ctrl+C.

### 7.2. Verificação de funcionamento

Noutra janela do PowerShell:

**Passo 6.1 — A API responde e exige autenticação:**

```
curl.exe -s -i http://localhost:5159/clinica
```

Resultado esperado: `HTTP/1.1 401 Unauthorized` e o corpo `{"code":401,"message":"Autenticação necessária.","detail":"É necessário apresentar um token JWT válido."}` (formato de erro da I058). Se em vez disso aparecer um erro 500 ou a ligação for recusada, ver os problemas P3, P4 e P10.

**Passo 6.2 — O Swagger abre:** no browser, `http://localhost:5159/swagger`. Resultado esperado: o Swagger UI com as 38 operações da API, agrupadas por controlador (o perfil `http` abre esta página automaticamente ao arrancar).

**Passo 6.3 — Iniciar sessão (EP-001)** com a conta do Administrador de demonstração (`admin@clinica-g08.test`; palavra-passe no ficheiro de credenciais, secção 2.3):

```
$sec = Read-Host -AsSecureString "Palavra-passe do admin@clinica-g08.test"
$pw = [Runtime.InteropServices.Marshal]::PtrToStringBSTR([Runtime.InteropServices.Marshal]::SecureS
tringToBSTR($sec))
$login = Invoke-RestMethod -Method Post -Uri http://localhost:5159/auth/login -ContentType "applica
tion/json" -Body (@{ email = "admin@clinica-g08.test"; password = $pw } | ConvertTo-Json)
Remove-Variable pw, sec
$login.utilizador
```

Resultado esperado: o perfil `Administrador` e um token em `$login.token`.

**Passo 6.4 — Pedido autenticado (EP-009):**

```
Invoke-RestMethod -Uri http://localhost:5159/clinica -Headers @{ Authorization = "Bearer $($login.t
oken)" } | Select-Object nome, nif
```

Resultado esperado: `nome` = `Clínica Veterinária G08`. A instalação está concluída.

Em alternativa aos passos 6.3 e 6.4, no Swagger: «POST /auth/login» → *Try it out* → copiar o `token` da resposta → botão **Authorize** → `Bearer <token>` → «GET /clinica» → 200.

### 7.3. Verificações adicionais (opcionais)

| Verificação | Comando, na pasta `src\backend` (`cd <raiz do repositório>\src\backend`) | Resultado esperado |
|---|---|---|
| Testes unitários | `dotnet test .\ClinicaVeterinaria.UnitTests\ClinicaVeterinaria.UnitTests.csproj` | 718 testes: 717 aprovados, 1 ignorado (`UT-TRV-007`, justificado) |
| Testes de integração (Docker Desktop ativo) | `dotnet test .\ClinicaVeterinaria.IntegrationTests\ClinicaVeterinaria.IntegrationTests.csproj` | 463 aprovados (3 a 5 minutos; cada classe cria o seu SQL Server em Docker) |
| Collection Postman | Ver o README de `03-artefactos-de-qualidade-de-produto\03.01-backend\03.01.05-postman` | 269 pedidos e 862 verificações sem falhas |

## 8. Repor o ambiente

Para voltar ao estado inicial (base com o esquema e os dados de demonstração), com a API parada:

1. Repetir o **passo 3.1** na pasta `src\backend` (o script apaga e recria a base) e o **passo 3.2**, mudando de pasta como nesse passo. Voltar depois à pasta `src\backend` (`cd ..\..\..\..\src\backend`).
2. Arrancar a API (passo 5.1) e repetir a verificação (secção 7.2).

Os *user secrets* mantêm-se. Para os apagar: `dotnet user-secrets clear --project .\ClinicaVeterinaria.Api\ClinicaVeterinaria.Api.csproj`. Na opção B, para remover o contentor: `docker rm -f sql-clinica`.

Para atualizar para uma versão nova do código sem perder dados: `git pull`, compilar (passo 1.4) e aplicar só as migrações novas com `dotnet ef database update --project ClinicaVeterinaria.Infrastructure --startup-project ClinicaVeterinaria.Api --context ClinicaDbContext` (com `--connection "<cadeia>"` se não for a cadeia por omissão).

## 9. Configuração por ambiente

| Ambiente | `ASPNETCORE_ENVIRONMENT` | Base de dados | Onde está a configuração e os segredos | Migrações |
|---|---|---|---|---|
| **Desenvolvimento local** (este manual) | `Development` (perfil `http`) | SQL Server 2025 local ou em Docker, base `ClinicaVeterinaria` | `appsettings.json`; `Jwt:Key` (e, se necessário, a cadeia) nos *user secrets* | Script de reposição da I062 (passo 3.1) |

| Ambiente | `ASPNETCORE_ENVIRONMENT` | Base de dados | Onde está a configuração e os segredos | Migrações |
|---|---|---|---|---|
| **Testes de integração** | `Development` (WebApplicationFactory) | Um SQL Server 2025 em Docker por classe de testes, criado e destruído pelo Testcontainers (I091) | Gerados pelos próprios testes em cada execução | Aplicadas pela infraestrutura de testes |
| **Pipeline** (GitHub Actions, I102 a I104) | `Development` na etapa da Collection Postman | SQL Server 2025 descartável no runner | Credenciais geradas em cada execução e escondidas nos registos; nenhum segredo no repositório | `dotnet ef database update` na etapa 7 |
| **Ambiente público de avaliação** (Azure, I104) | `Production` | Azure SQL Database `clinica-g08-avaliacao` | Definições do App Service (`ClinicaDb`, `Jwt__Key`, `Swagger__Ativo = true`, `BaseDados__AplicarMigracoesNoArranque = true`); perfil de publicação no ambiente `avaliacao` do GitHub | Aplicadas pela API no arranque |

### 9.1. Ambiente público de avaliação

A instalação, a configuração e a reposição do ambiente público estão no README da I104 (`/projeto/04-artefactos-tecnicos/04.16-ci-cd/m3-v03/README.md`, secções 2, 3 e 6). Atenção: **não usar o script de reposição da I062 na base do Azure** — apaga a base, que perderia a oferta gratuita; a reposição pública desfaz e volta a aplicar as migrações, como descrito nesse README.

## 10. Problemas frequentes

| Ref. | Sintoma | Causa | Solução |
|---|---|---|---|
| P1 | `A compatible .NET SDK was not found` ou erro sobre o `global.json` | .NET SDK 10 não instalado | Instalar o .NET SDK 10 (série 10.0) e abrir uma janela nova do PowerShell |
| P2 | `dotnet ef` não é reconhecido | Ferramenta não instalada ou a pasta das ferramentas globais não está no PATH | Passo 1.3; abrir uma janela nova do PowerShell |
| P3 | A API termina ao arrancar com «A chave JWT (Jwt:Key) não está configurada» ou «tem de ter pelo menos 32 bytes» | Falta a chave JWT ou é inválida | Passo 4.1 |
| P4 | `A network-related or instance-specific error occurred` ou `Login failed` no passo 3.1 ou nos pedidos à API | SQL Server parado, instância com nome, porta errada ou palavra-passe errada | Confirmar o serviço «SQL Server (MSSQLSERVER)» ou o contentor (`docker ps`); ajustar a cadeia (por exemplo, `Server=localhost\SQLEXPRESS`) e guardá-la outra vez no passo 4.2 |
| P5 | Erro 500 no login ou `The ConnectionString property has not been initialized` | A cadeia de ligação não está nos *user secrets* (em Development fica vazia) | Passo 4.2 |
| P6 | `The certificate chain was issued by an authority that is not trusted` | O SqlClient e o `sqlcmd` 18 cifram a ligação por omissão | `TrustServerCertificate=True` na cadeia; `-ConfiarCertificado` nos scripts; `-C` no `sqlcmd` |
| P7 | «O sqlcmd não foi encontrado» no passo 3.2 | `sqlcmd` não instalado | Instalar as SQL Server Command Line Utilities ou o `go-sqlcmd` e abrir uma janela nova |
| P8 | `... cannot be loaded because running scripts is disabled on this system` | Política de execução do PowerShell | Usar `powershell -ExecutionPolicy Bypass -File <script>`, como nos passos 3.1 e 3.2 |

| Ref. | Sintoma | Causa | Solução |
|---|---|---|---|
| P9 | «Faltam dados de referência (Clinica ou as espécies Cão e Gato)» no passo 3.2 | A base não foi criada pelas migrações | Repetir o passo 3.1 antes do 3.2 |
| P10 | `Failed to bind to address http://127.0.0.1:5159: address already in use` | Outra instância da API ou outro programa na porta 5159 | Fechar a outra instância (Ctrl+C) ou arrancar com `--urls http://localhost:5160` |
| P11 | 401 «É necessário apresentar um token JWT válido» num pedido | Pedido sem token ou com token expirado (8 h) | Iniciar sessão (passo 6.3) e enviar `Authorization: Bearer <token>` |
| P12 | 401 «Credenciais inválidas.» ao iniciar sessão, ou 403 «A conta está bloqueada. Contacte a clínica.» | Palavra-passe errada; a conta `rececao.bloqueada@clinica-g08.test` está bloqueada de propósito | Usar as palavras-passe do ficheiro de credenciais (secção 2.3) e outra conta |
| P13 | Testes de integração falham com `Docker is either not running or misconfigured` | Docker Desktop parado | Iniciar o Docker Desktop (contentores Linux) e repetir |
| P14 | Acentos trocados nos registos ao usar `Get-Content` no Windows PowerShell 5.1 | Leitura sem UTF-8 | `Get-Content <ficheiro> -Encoding UTF8` |

## 11. Validação do manual

O manual é validado passo a passo, num ambiente limpo, por um elemento do grupo diferente do autor (critério 4 da I038). O registo da validação (quem, quando, máquina e versões, resultado de cada passo e desvios encontrados) fica no ficheiro `m3-s05-i038-AAAAMMDD-registo-validacao-v01.md`, nesta pasta, e os desvios encontrados são corrigidos neste manual antes da versão final em PDF.